

RAG 检索增强生成技术极简入门手册

RAG 检索增强生成技术极简入门手册

第 1 页 基础定义与解决痛点

1.1 RAG 是什么

RAG 全称 **Retrieval-Augmented Generation**，**检索增强生成**，2020 年由 Meta 提出，是给大模型搭配外部知识库的基础架构。

核心逻辑：先检索私有 / 实时文档，再把检索内容作为上下文喂给大模型生成答案，相当于给大模型开卷考试。

1.2 纯大模型的 4 大缺陷（RAG 针对性解决）

1. **知识截止**：模型训练数据固定，无法获取最新资料
2. **幻觉问题**：无依据编造专业事实，可信度低
3. **私有数据盲区**：企业内部文档、本地文件模型无法读取
4. **超长上下文限制**：无法一次性灌入全部知识库

1.3 RAG 核心优势

- 低成本：不用重训大模型，仅更新文档即可更新知识
- 可溯源：回答附带原文片段，方便校验真伪
- 轻量化：本地小模型也能实现专业问答
- 灵活迭代：新增 / 删除文档无需改动模型权重

第 2 页 RAG 标准完整流水线（两大阶段）

阶段 1：离线索引构建（提前一次性执行）

1. **文档加载**：读取 PDF、Word、TXT、Markdown 等文件
2. **文本分块 Chunk**：长文本切割为小段，设置重叠文本避免语义断裂（常用 chunk_size=300~800）
3. **向量化 Embedding**：文本转为数字向量，语义相近向量距离更近，中文推荐 BGE、m3e
4. **向量入库**：存入向量数据库（Chroma、Milvus、FAISS、Redis Vector）建立索引

阶段 2：在线问答推理（用户提问实时执行）

- 1. 用户 Query 输入
- 2. 查询向量化：问题转为向量
- 3. 相似检索：向量库召回 Top-K 匹配片段（粗召回）
- 4. 重排序 Rerank：交叉编码器筛选最相关片段（降噪）
- 5. Prompt 拼接：问题 + 检索原文整合为提示词
- 6. LLM 生成回答：基于真实文档输出答案，标注来源

第 3 页 核心组件与 RAG vs 微调对比

3.1 三大核心模块

- 1. 嵌入模型 Embedding：语义转换工具，负责文本向量化
- 2. 向量检索器 Retriever：快速匹配相似文本，分为关键词检索 (BM25)、语义向量检索
- 3. 大生成模型 LLM：整合检索内容输出通顺答案

3.2 RAG 与模型微调对比表

对比维度	RAG 检索增强	LLM 微调
原理	读取外部知识库	修改模型内部参数
更新知识	新增文档即可，秒级生效	重新训练，成本极高
硬件成本	低，普通 CPU 可运行	高，需高端 GPU
可解释性	高，每条回答附带原文	低，无法追溯依据
适用场景	知识库、企业文档、实时资讯	风格统一、固定简短任务

第 4 页 简易落地流程与常见优化

4.1 最小可运行 RAG 开发步骤（Python）

- 1. 安装依赖：langchain、chromadb、ollama、pypdf
- 2. 加载本地文档，执行文本分块
- 3. 调用 Embedding 模型生成向量存入本地向量库
- 4. 用户提问→检索片段→拼接 prompt→调用本地大模型生成回答

4.2 主流优化方案（解决低质问答）

1. **混合检索**：BM25 关键词检索 + 向量检索双路召回
2. **查询改写**：扩写、拆分用户模糊问题，提升召回精度
3. **多轮对话 RAG**：存储历史对话，理解上下文提问
4. **分层分块**：大段摘要 + 细粒度片段双层检索
5. **过滤降噪**：过滤无关、重复、低质量检索片段

4.3 典型落地场景

企业知识库问答、本地文档助手、行业专业手册查询、客服智能 FAQ、论文文献检索问答

（注：部分内容可能由 AI 生成）